

Question 1

Ans: a) $\Theta(g(n))$:

$f(n) = \Theta(g(n))$ if and only if there exists positive constants

C_1, C_2, n_0 such that

$$0 \leq C_1 g(n) \leq f(n) \leq C_2 \cdot g(n) \text{ for all } n \geq n_0.$$

b) Word RAM cost measures: running time (number of primitive/word operations) and space (number of memory words or cells used)

Q2.

Ans: Outer loop count: i takes values $1, 2, 4, \dots, n$, which equal $2^0, 2^1$, up to 2 raised to the power ($\log$ base 2 of n). So it runs $(\log$ base of $n) + 1$ times.

$$\text{let } m = (\log \text{ base } 2 \text{ of } n) + 1.$$

b. ~~Word RAM cost measures:~~

Cost / Repetitions / Total table:

Statement	Cost	Repetitions	Total
int sum=0, i, j;	1	1	1
for (i=1; i<=n; i*=2)	2	$m+1$	$2m+2$
for (j=0; j<n; j++)	2	$m \text{ times } (n+1)$	$m(2n+2)$
sum = sum + A[j];	2	mn	$2mn$
return sum;	1	1	1

c. Exact $T(n)$:

$$T(n) = 1 + (2m+2) + m(2m+2) + 2mn + 1$$

$$= 4mn + 4m + 4$$

Substituting $m = (\log_2 n) + 1$:

$$T(n) = 4n \cdot [(\log_2 n) + 1] + 4 \cdot [(\log_2 n) + 1] + 4$$

$$= (4n+4) \cdot (\log_2 n) + 4n + 8$$

D. Dominant term is $4n$ times $(\log_2 n)$, so $T(n) = \Theta(n \log n)$.

choose $c_1 = 4$

$c_2 = 8$

$n_0 = 2$

for $n \geq 2$.

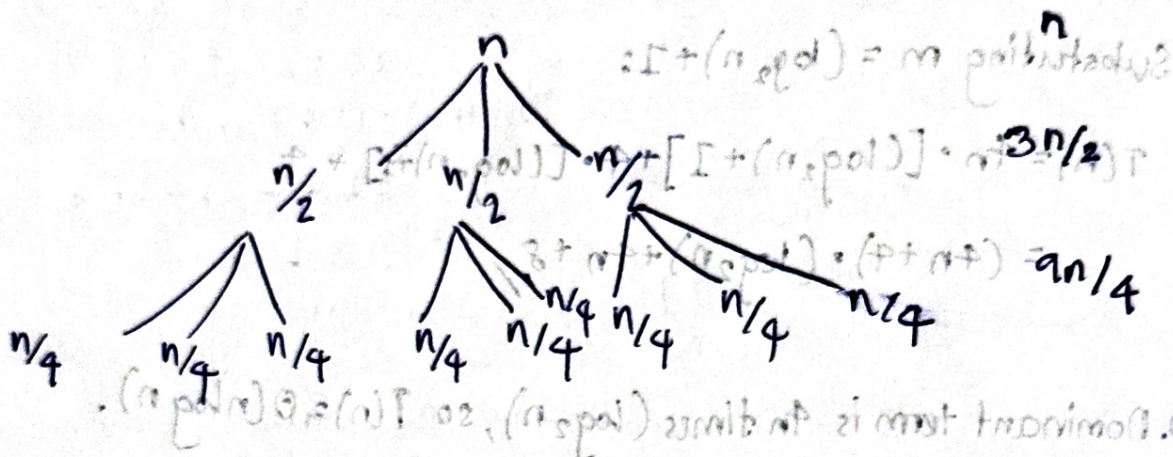
$$4n (\log_2 n) \leq (4n+4) \cdot (\log_2 n) + 4n + 8 \leq 8n \cdot (\log_2 n)$$

Q3.

Ans: Recursion Tree for $T(n) = 3 \cdot T(n/2) + n$

a.

$$T(n) = 3 \cdot T(n/2) + n$$



b. General level i : 3^i nodes each size of $n/2^i$.
each doing $n/2^i$ work.

$$\text{Level cost} = 3^i (n/2^i)$$

$$(n/2^i) \cdot 3^i = n \cdot (3/2)^i$$

c. Height & leaves: Tree ends when $n/2^i = 1$.

$$\text{height} = \log_2 n$$

$$\text{leaves} = 3^{\log_2 n}$$

$$= n^{\log_2 3} //$$

d. Summing levels:

$\sum_{i=0}^{\log_2 n - 1} n(3/2)^i$ is an increasing geometric series, so it's dominated by its last term, which is in order of the leaf count. Gives a total of $\Theta(n^{\log_2 3})$ after evaluating.

e. Since the branching factor (3) exceeds the size reduction factor (2), the leaves dominate the total cost. Contrast: $T(n) = 2T(n/2) + n$ has $a=b=2$, so every level cost exactly $n \rightarrow$ all levels equal, giving $\Theta(n \log n)$. $T(n) = 2T(n/2) + n^2$ has $f(n)$ growing faster than the tree branches $\rightarrow$ root dominates, giving $\Theta(n^2)$.

Q4.

Ans: The falling line = $4c(n/2)^2 + n \leq cn^2$

This is false for every $c > 0$ and $n > 0$. - We are claiming a positive number (n) doesn't increase cn^2 when added to it. The constant c is fixed once at the start of the proof; you can't invoke "n is lower-order" to make an additive term vanish against a fixed coefficient cn^2 . That only works when there is a term to subtract against.

b. Strengthened Hypothesis: $T(n) \leq cn^2 - dn$ with $c > 0, d > 0$.
 subtracting a lower-order term gives the induction "steak" to absorb the $+n$.

c. Inductive Step.

$$T(n) = 4T(n/2) + n$$

$$\leq 4 \left[c \left(\frac{n}{2} \right)^2 - d \left(\frac{n}{2} \right) \right] + n$$

$$= 4 \left(\frac{cn^2}{4} - \frac{dn}{2} \right) + n$$

$$= cn^2 - 2dn + n$$

$$= cn^2 - (2d - 1)n$$

what we want:

$$cn^2 - (2d - 1)n \leq cn^2 - dn$$

$$(2d - 1)n \geq dn$$

$$d \geq 1$$

d. Base case: $n_0 = 1$

$$T(1) \leq c(1)^2 - d(1) + \left(\frac{1}{2} \right) = 1 - d + \frac{1}{2} = \frac{3}{2} - d$$

$$1 \leq c - d$$

$$\text{when } d = 1 \quad T(1) = 1 \leq 2 - 1$$

$$c = 2$$

$$= 1$$

so the strengthened claim holds

$$T(n) \leq 2n^2 - n \quad \text{for } n \geq 1$$

$$\Rightarrow T(n) = O(n^2)$$

Q5.

Ans: a) Pseudocode:

MINMAX (A, lo, hi):

if $hi - lo == 0$:

return (A[lo], A[lo])

if $hi - lo == 1$:

if $A[lo] < A[hi]$:

return (A[lo], A[hi])

else:

return (A[hi], A[lo])

Comparison total

mid = $(lo + hi) // 2$

(min 1, max 1) = MINMAX (A, lo, mid)

(min 2, max 2) = MINMAX (A, mid + 1, hi)

if min 1 < min 2: minVal = min 1 else minVal = min 2 # Comp

if max 1 > max 2: maxVal = max 1 else maxVal = max 2 # Comp 2

return (minVal, maxVal)

b) Recurrence: $C(n) = 2C(n/2) + 2^n + \left(\frac{n/2}{2} - \frac{n/2}{2}\right) \cdot 1 =$

with base case $C(2) = 1$

$C(1) = 0$.

c) Solving by unrolling

$$C(n) = 2C\left(\frac{n}{2}\right) + 2$$

$$= 2 \left[2C\left(\frac{n}{4}\right) + 2 \right] + 2$$

$$C(n) = 2C\left(\frac{n}{2}\right) + 2$$

$$2 \left[2C\left(\frac{n}{4}\right) + 2 \right] + 2 = 4C\left(\frac{n}{4}\right) + 4 + 2$$

$$2 \left[2C\left(\frac{n}{8}\right) + 2 \right] + 4 + 2 = 4C\left(\frac{n}{8}\right) + 8 + 4 + 2$$

$$11 \left(\frac{n}{8} \right) = (n) \cdot 11$$

$$1 - 5 \geq 1 - (1) +$$

$$1 - 5 \geq 1 - (1) +$$

$$1 - 5 \geq 1 - (1) +$$

After k expansions:

$$C(n) = 2^k \cdot c\left(\frac{n}{2^k}\right) + 2^{k+1} - 2$$

we get this from:

$$C(n) = 2^k \cdot c\left(\frac{n}{2^k}\right) + (2^k + 2^{k-1} + \dots + 4 + 2)$$

Base case substitution:

when $\frac{n}{2^k} = 2$ $C(n) = 2^{\log_2 n - 1} \cdot C(2) + 2^{\log_2 n} - 2$

$$k = \log_2 n - 1$$

$$= \frac{n}{2} \cdot 1 + n - 2$$

$$= \frac{3n}{2} - 2$$

$$C(n) = \Theta(n) //$$

d)	n	Measured comparisons	Predicted by closed form	$2n - 2$ (Two-sum method)
	8	10	10	14
	64	94	94	126
	512	766	766	1022
	4096	6142	6142	8190

e) Measurement matches prediction exactly. Comparisons are discrete, Everytime we run MINMAX on n elements, it does the exact number of comparisons the recurrence says. It does not depend on the actual values in the array. Also, since we only tested with n as power of 2, there were no floor or ceiling terms in the recursion. That means the closed form solution works out perfectly, not just approximately.

Practice Questions

Q1.

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$a = 2$$

$$b = 2$$

$$f(n) = n$$

$$g(n) = n^{\log_b a}$$

$$g(n) = n^{\log_2 2}$$

$$g(n) = n$$

$$g(n) = f(n)$$

case 2 //

$$f(n) = O(n^{\log_2 2})$$

$$T(n) = O(\log_2^2 n \cdot \log^k n)$$

$$T(n) = O(n \cdot \log^k n) //$$

Q2. Non recursive cost: the for loop runs exactly n times

$$\text{Recurrence: } T(n) = 2T(n/2) + \Theta(n), \quad T(1) = \Theta(1)$$

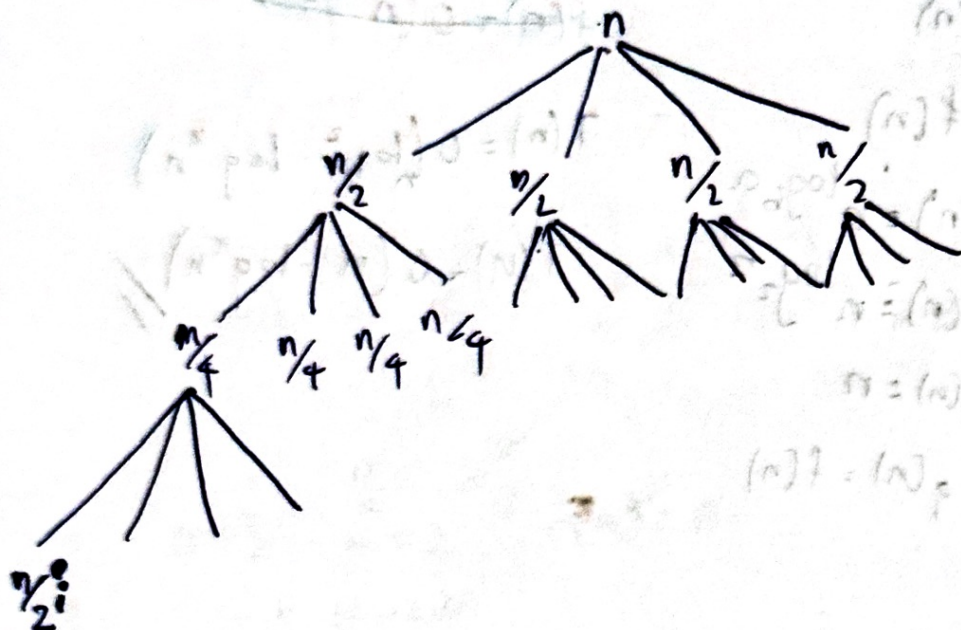
$$\Theta(n) = \Theta(n \log_2^2)$$

$$T(n) = \Theta(n \log n) //$$

If the loop were moved before both recursive calls, the answer does not change. The master theorem only cares about the total amount of non-recursive work done per call and number of recursive calls, not the other order in which that work happens. Moving the loop to the top still contributes exactly $\Theta(n)$ non-recursive work per call and the two recursive calls are unaffected.

Q3

$$T(n) = 4T(n/2) + n$$



recursion tree

$$(n) \Theta + \left(\frac{n}{2}\right) 13 = (n) 17$$

$$(n) 17 + \left(\frac{n}{2}\right) 17 = (n) 17$$

$$2n$$

$$2 = n$$

$$4n \quad 2 = n$$

$$n = (n) 17$$

$$\frac{4^i}{2^i} \cdot n = 2^i n$$

Total cost leaf cost height = $\log_2 n$

$$T(n) = \sum_{i=0}^{\log_2 n} 2^i \cdot n + n^2 \Theta + T(n) = \Theta(n^2) //$$

$$= n \cdot (2^{\log_2 n + 1} - 1)$$

$$\approx 2n^2 //$$

$\therefore$ The leaf dominates the cost.

Q4. For fixed i , j takes values $1, 1+i, 1+2i, \dots$ up to n , giving

$\left\lceil \frac{n-1}{i} \right\rceil + 1$ iterations - approximately n/i .

$$\text{count} = \sum_{i=1}^n \left(\left\lceil \frac{n-1}{i} \right\rceil + 1 \right) \approx \sum_{i=1}^n \frac{n}{i} = n \sum_{i=1}^n \frac{1}{i} = n H_n$$

$$H_n = \Theta(\log n)$$

$$\therefore \text{result} = \Theta(n \log n)$$

$$Q8. T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2)$$

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$a = 7$$

$$b = 2$$

$$f(n) = \Theta(n^2)$$

$$f(n) = g(n)$$

$$n^2 < n^{2.80}$$

$$\text{case 1 } T(n) = \Theta(n^{2.807}) //$$

Q11. Master theorem

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$a = 3 \quad b = 2 \quad f(n) = n$$

$$g(n) = n \log a$$

$$= n \frac{\log 3}{\log 2}$$

$$= n^{1.586} //$$

compare

$$f(n) = g(n)$$

Case 2

$$T(n) = \Theta(n^{1.586}) //$$

Q.17 Master theorem

$$T(n) = 2T(n/2) + n^2$$

$$T(n) = 2a(n/b) + f(n)$$

$$a=2 \quad b=2 \quad f(n)=n^2$$

$$g(n) = n \log a b$$

$$= n \log_2 2$$

$$= n //$$

Q.51. Master theorem

$$T(n) = 16T(n/4) + n^2$$

$$a=16 \quad b=4 \quad f(n)=n^2$$

$$g(n) = \frac{n \log a}{\log b}$$

$$= n^2 //$$

Q.57 Master theorem

$$T(n) = 3T(n/4) + n \log n$$

$$a=3 \quad b=4 \quad f(n)=n \log n$$

$$g(n) = \frac{n \log a}{\log b}$$

$$= \frac{n \log 3}{\log 4}$$

$$= n^{0.792} //$$

compare

$$f(n) = g(n)$$

$$n^2 > n$$

case III

$$T(n) = \Theta(n) //$$

compare

$$f(n) = g(n)$$

$$n^2 = n^2$$

case I

$$(n)^1 + (n)^1 + \dots = (n)^1$$

compare

$$f(n) = g(n)$$

$$n \log n \approx n^{0.792}$$

Q. 58

- outer i run from $n/2$ to n , i.e. $(n/2 + 1)$ iterations
- middle $n/2$ iterations, independent of i
- inner 1 double. $\lceil \log_2 n \rceil + 1$ iterations

$$\text{count} = (n/2 + 1) \cdot n/2 \cdot (\log_2 n) + 1 \approx n^2/4 \log_2 n$$

$$\Theta(n^2 \log n) \text{ (machine ratio} \rightarrow 1/4)$$